**Master in Life Sciences**

a Joint-Venture between HES-SO, ZHAW, FHNW and BFH

2024

# Parameter Estimation and Curve Fitting

# for a

# Logistic Regression Classifier

---

Basil Imoberdorf – HES-SO Wallis/Valais

---

**CO3 – Optimization and Bio-Inspired Algorithms**

Lecturers: Prof. Dr. Thomas Ott & Dr. Ahmad Aghaebrahimian

## Table of content

1 Defining the optimization problem ..... 2
2 Defining the optimization algorithms ..... 3
2.1 Gradient Descent ..... 3
2.2 Simulated Annealing ..... 4
3 Results ..... 5
3.1 Gradient Descent ..... 5
3.2 Simlated Annealing ..... 6
4 Conclusion ..... 6
5 Literature ..... 7
6 Appendix ..... 8
6.1 Additional figures ..... 8
6.2 Introduction to classification with logistic regression ..... 9
6.3 Code ..... 10

# 1 Defining the optimization problem

The goal of this project was to estimate the parameters for a logistic regression classifier that can distinguish two types of flowers, namely Iris Virginica and Iris Versicolor from the well-known Iris flower data set.[1] More details on the mathematics of logistic regression classifiers can be found in the annex.

The fitting of a binary classifier with only one input dimension, namely the petal length of the leaves, was motivated to be able to visualize the loss landscape and the dynamics of the optimization algorithms. If there are more than two classes the number of parameters to estimate is $k(n+1)$, where $k$ is the number of classes and $n$ the number of input dimensions. It is therefore only possible to directly visualize the binary case.

To make it more interesting the Sum of Squares Error (SSE) was applied as objective function instead of the binary cross-entropy loss. The general form of the SSE is given in equation 1.

$$SSE(\hat{y}) = \frac{1}{2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \qquad \text{Eq. 1}$$

SSE penalizes the difference between the predicted value, $\hat{y}_i$, and the true value or label, $y_i$. The difference is squared to prevent positive and negative differences from canceling each other out and to penalize bigger differences more than small ones. The factor $1/2$ is used for convenience when calculating the derivative.

The prediction is in the case of univariate binary logistic regression given by $\hat{y}_i = \frac{1}{1+e^{-(ax_i+m)}}$.

The optimization problem can then be formulated as follows:

$$\min_{a,m} SSE(a,m) = \min_{a,m} \frac{1}{2}\sum_{i=1}^{n}\left(y_i - \frac{1}{1+e^{-(ax_i+m)}}\right)^2 \qquad \text{Eq. 2}$$

Where $a$ and $m$ are the parameters to be estimated. The SSE function is smooth and continuous and has only one minimum. Furthermore, the problem is unconstrained meaning that the parameters can theoretically take on any value. However, for the algorithm to find a solution the distributions of the two classes must overlap a little, because if the classes are linearly separable, the optimal value for the slope, $a$, would be ∞ which is hard to converge to. In such cases, a regularization must be applied so that the optimal $a$ becomes finite.[2]

Figure 1 depicts the resulting loss landscapes when logistic regression is applied to the mentioned subset of the Iris data set. Remark: Mean cantering the input was necessary to remove distortions and to fan out the landscape to give it a pleasing look.

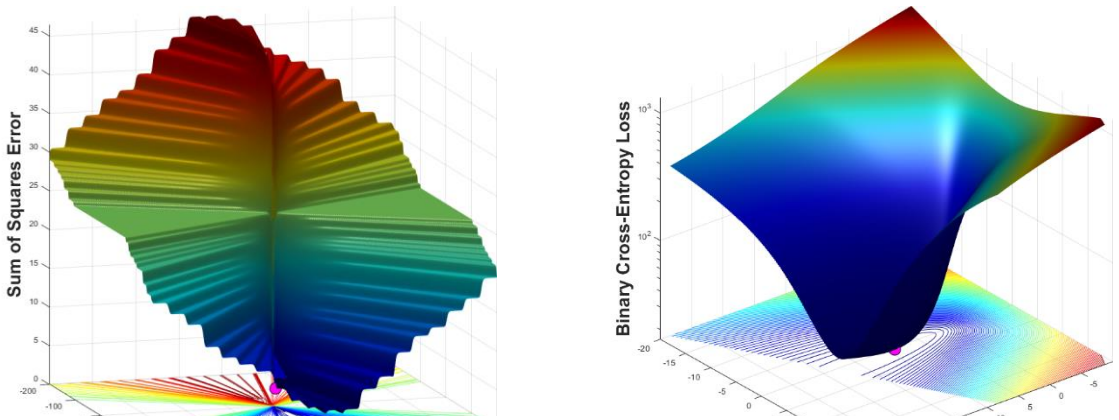



*Figure 1: Loss landscapes for SSE (left) and binary cross-entropy loss (right) on a subset of the iris data set are shown as surface plots with a corresponding contour plot below and the optimum indicated with a magenta scatter point. The parts of the BCE landscapes that are cut out are regions where the floating-point precision is no longer sufficient, and the sigmoid function evaluates to 0, which is undefined in the logarithm used for cross-entropy loss. (z-axis in logarithmic scale)*

SSE shows an intricate loss landscape that has "folds" depending on the number of observations. With a relatively small number of observations, these folds form step-function-like structures in the loss landscape, which become increasingly smooth as the number of observations increases.

# 2 Defining the optimization algorithms

In the following section the two methods used to solve the problem are described. These are Gradient Descent (GD) and Simulated Annealing (SA).

## 2.1 Gradient Descent

The main idea behind gradient descent is to follow the steepest slope of a function at a given point until one reaches an extremum. This implies that the algorithm can only be applied to smooth functions that are continuous on the interval of interest.
Since the previously defined objective function is smooth and continuous we can calculate the gradient. The gradient is a vector containing all the partial derivatives of the objective function with respect to the parameters. For the problem at hand GD can then be written as a system of equations. To keep it short the derivation of the partial derivatives is not shown but it can readily be found by applying the chain rule to the objective function.

$$a_{i+1} = a_i - \eta_i \frac{\partial SSE}{\partial a} = a_i - \eta_i \sum_{i=1}^{n} \hat{y}_i(\hat{y}_i - y_i)(1 - \hat{y}_i)x_i \qquad \text{Eq. 3}$$

$$m_{i+1} = m_i - \eta_i \frac{\partial SSE}{\partial m} = m_i - \eta_i \sum_{i=1}^{n} \hat{y}_i(\hat{y}_i - y_i)(1 - \hat{y}_i) \qquad \text{Eq. 4}$$

Here, $\eta_i$ is the learning rate at iteration step $i$. It scales the gradient to efficiently move through the loss landscape. Typically, the learning rate is set to decrease over time to allow the algorithm to converge on a solution.

Since the objective function features regions where the gradient almost vanishes and regions where the gradients are very step it is hard to tune the learning rate by hand as it tends to overshoot or undershoot at some point. To mitigate this problem a modified variant of GD called Normalized Gradient Decent (NGD) was also implemented. The idea in NGD is to uncouple the direction of the gradient from its magnitude. By only using the information of the direction it is much easier to tune the learning rate such that the step size is robust no matter the magnitude of the underlying gradient.[3] To achieve this, each component of the gradient is rescaled by the magnitude of the gradient, $\|\nabla SSE\|_2$, at each iteration step. Then the step size becomes 1 and can be tuned by the learning rate as desired. For the given problem normalized gradient decent can be written as follows:

$$a_{i+1} = a_i - \frac{\eta_i}{\|\nabla SSE\|_2} \frac{\partial SSE}{\partial a} \qquad \text{Eq. 5}$$

$$m_{i+1} = m_i - \frac{\eta_i}{\|\nabla SSE\|_2} \frac{\partial SSE}{\partial m} \qquad \text{Eq. 6}$$

With the Euclidean norm (magnitude): $\|\nabla SSE\|_2 = \sqrt{\left(\frac{\partial SSE}{\partial a}\right)^2 + \left(\frac{\partial SSE}{\partial m}\right)^2}$ Eq. 7

A suitable function to tune the step size has been found in equation 8, a modified variant of the sigmoid function. By introducing three new hyperparameters, $\lambda_{max}$, $\alpha$, and $\gamma$ it is possible to shape an efficient descent profile for the problem at hand.

$$\eta_i = \frac{\lambda_{max}}{1 + \exp\left(\alpha\left(i - \frac{N}{\gamma}\right)\right)} \qquad \text{Eq. 8}$$

## 2.2 Simulated Annealing

Simulated Annealing is a more sophisticated version of Mont Carlo sampling which does not depend on gradients. It is however sensitive to the initial guess for the parameters and to the hyperparameters. Due to its random nature, it typically needs many more iterations to converge on the optimum. In contrast to GD, it can handle non-convex functions with many local extrema.

To solve the problem at hand the following form of a SA algorithm was implemented:

1. Chose initial parameters, $a_0,\ b_0$ and evaluate $SSE_0(a_0,\ b_0)$
2. Chose new parameters , $a_{trial},\ b_{trial}$, by adding scaled gaussian noise.
   - $a_{trial} = a_i + p_i \cdot rand_n()$
   - $b_{trial} = b_i + p_i \cdot rand_n()$
   -
3. Evaluate loss: $SSE_{i+1}(a_{trial}, b_{trial})$
   - **if** $SSE_{i+1} \leq SSE_i$
     - $a_{i+1} = a_{trial}$
     - $b_{i+1} = b_{trial}$
   - **else if** $\exp\left(-\frac{SSE_i - SSE_{i+1}}{T_i}\right) \geq rand_u()$
     - $a_{i+1} = a_{trial}$
     - $b_{i+1} = b_{trial}$
   - **else**
     - $a_{i+1} = a_i$
     - $b_{i+1} = b_i$
     - $SSE_{i+1} = SSE_i$
4. Increment $i = [0,1,\ 2, \ldots, n]$ and return to step 2.

Where $rand_n()$ is a function that draws a number from the standard normal distribution and $rand_u()$ is a function that draws a number between 0 and 1 form a uniform distribution. $p_i$ and $T_i$ are functions of the same form as equation 8. $p_i$ scales the random gaussian. At first the algorithm takes big steps to explore and after some time once he hopefully has found the optimum, he can home in by decreasing the step size. The same is true for the $T_i$ only that it describes more the willingness to explore.

It has to be noted that for objective functions where it is known that there is only one minimum or maximum, omitting the exploration promoting Boltzmann acceptance criterion can improve the rate of convergence as the algorithm moves more directly to the optimum.

# 3 Results

## 3.1 Gradient Descent

Native GD works only if it is initialized near the optimum. Figure 2 depicts a run of GD where it initialized with $\eta_i = 0.1$ at $a_0 = -7$ and $b_0 = -1$ which is close enough to the optimum to converge. A maximum of 400 iterations was used as termination criterion.

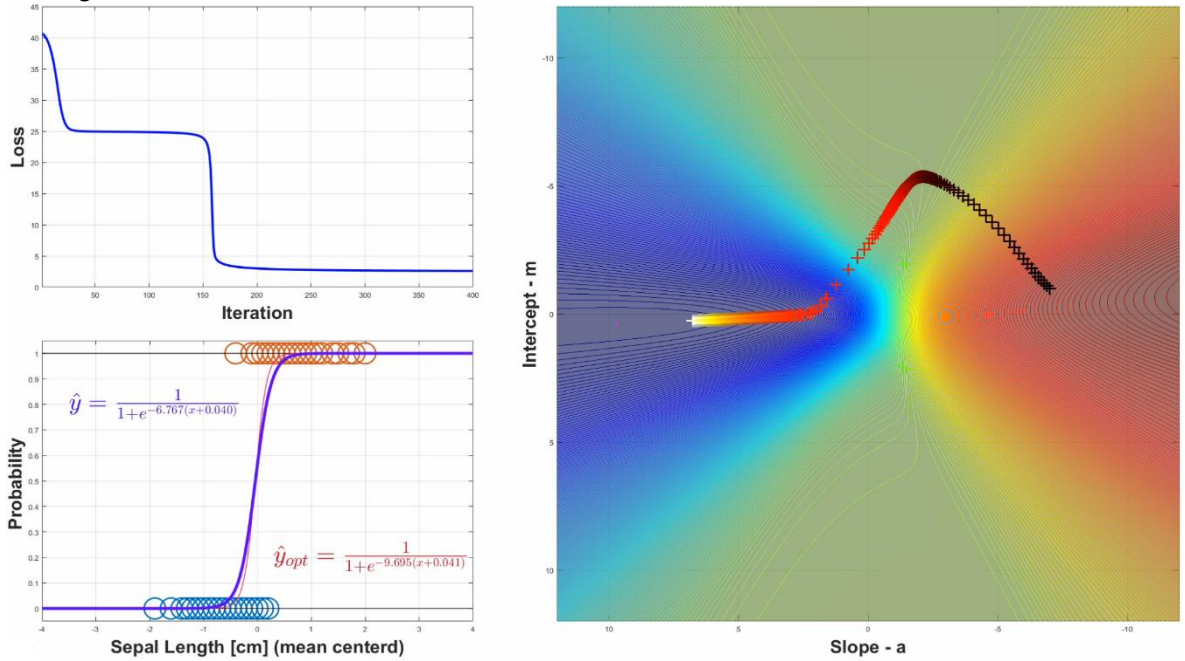



*Figure 2: Visualization of a GD run. Right Side: Contour plot of the loss landscape with markers for the current loss at each iteration step indicating the path used by the algorithm. Top Left: Evolution of the objective function value aka loss. Bottom Left: Final form of fitted curve is shown in blue and the optimal curve in magenta. The circles represent the instances.*

In the Appendix an example can be found that shows how GD gets stuck if initialized further away. Figure 3 shows the result of NGD which can traverse the plateau and reach the minimum efficiently after 100 iterations. The algorithm was initialized at $a_0 = -7$ and $b_0 = -1$

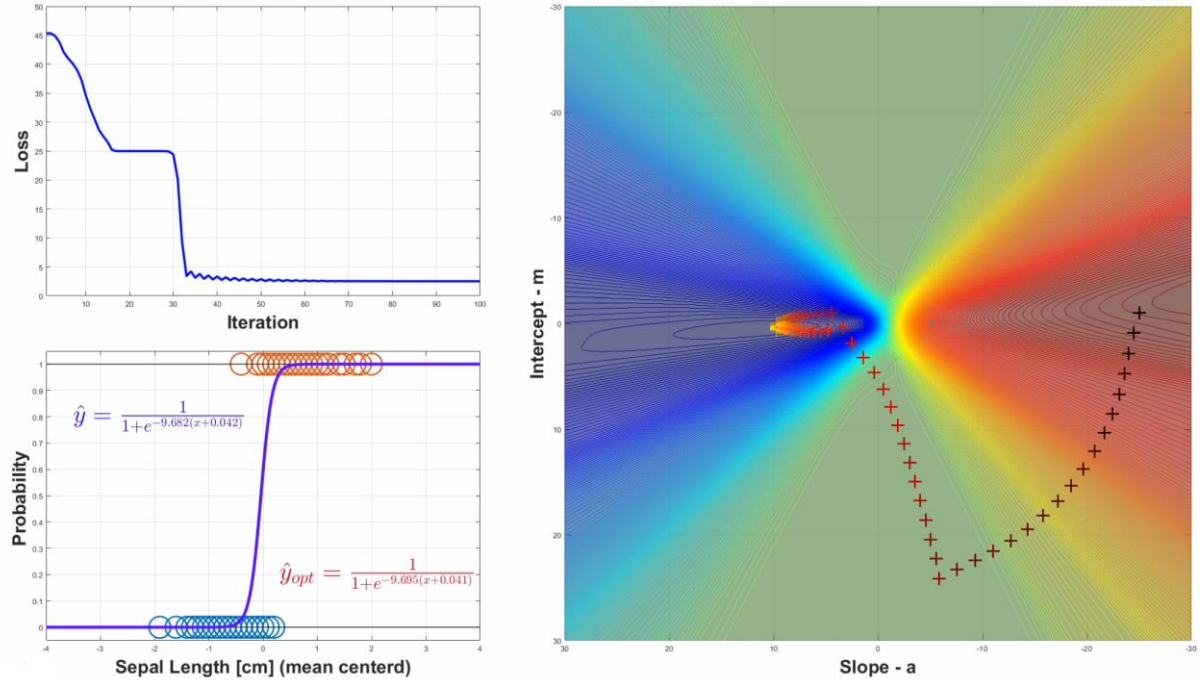



*Figure 3: Visualization of a NGD run. In contrast to GD NGD can traverse the plateau and home in to the optimum. Little oscilations can be observed as soon as the algorithm encounters steep gradients. The function to decrease the learning rate manges succesfully to shrink the step size and prevent divergence.*

## 3.2 Simulated Annealing

Simulated Annealing was also tested successfully. Figure 4 depicts a run which was initialized at $a_0 = -146$ and $b_0 = -10$ which is much further away from the optimum than what was tested in NGD. Nevertheless, the algorithm was able to converge but it needs usually more iterations than NGD.

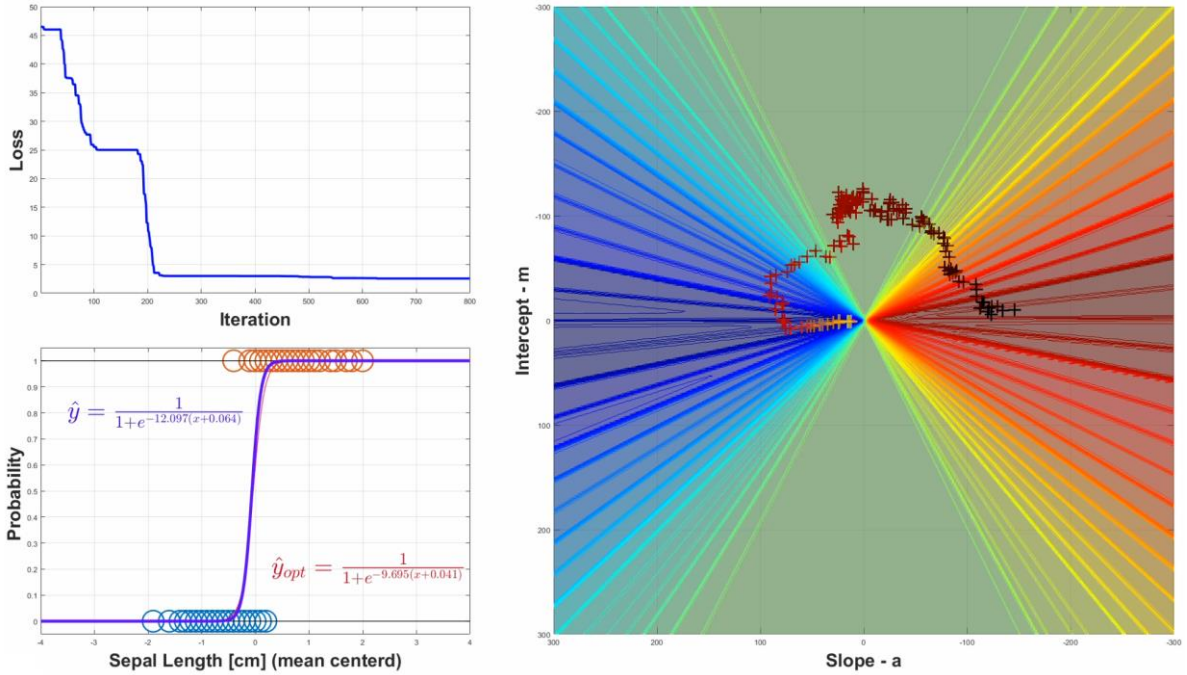



*Figure 4: Visulization of a Simualted Annealing run. As can be seen the initializtion is much further away from the ioptimum than what was used for Gradient Descent.*

To compare the performance of SA and (N)GD with a professional established algorithm, the downhill simplex algorithm also called Nelder-Mead method was implemented using the minimize function from the SciPy library. It was found that the Nelder-Mead algorithm converges very efficiently, typically in less than 100 iterations, but the initial guess needs to be fairly close to the optimum otherwise the algorithm cannot converge anymore. With $a_0 \leq -20$ and $b_0 \geq +20$, the Nelder-Mead algorithm fails. This stands in contrast to the SA and NGD algorithms which both can converge even if initialized much further away.

# 4 Conclusion

Gradient Descent is not robust. It depends highly on the initial guess for the parameters. It gets stuck in shallow gradients and overshoots on steep gradients. Tuning the learning rate is very hard and requires a good intuition of the underlying loss landscape. Normalizing the gradient allows for more control. With normalized gradients it is much easier to come up with a function for the learning rate that leads to good results without knowing exactly the form of the loss landscape. This makes NDR more robust. Furthermore, NGD can be initialized far from the optimum. However, depending on the objective function it might be computationally expensive to calculate the gradient.

Simulated Annealing is more robust in the sense that it can explore the loss landscape using the Boltzmann acceptance criterion and it also does not need gradient information. In terms of time, an iteration of SA is much faster than GD, as no gradients must be calculated. However, due to its random nature, SA often needs for consistency many more iterations than GD to converge to the optimum. By careful tuning of the step size and temperature one can initialize SA very far from the optimum and still converge. Lastly, removing the Boltzmann acceptance criterion can improve convergence if it is known that there is only one optimum. In that case exploring might not be that useful.

# 5 Literature

[1] FISHER, R.A. (1936), THE USE OF MULTIPLE MEASUREMENTS IN TAXONOMIC PROBLEMS. Annals of Eugenics, 7: 179-188. https://doi.org/10.1111/j.1469-1809.1936.tb02137.x

[2] Mohammad Ali Mansournia, Angelika Geroldinger, Sander Greenland, Georg Heinze, Separation in Logistic Regression: Causes, Consequences, and Control, American Journal of Epidemiology, Volume 187, Issue 4, April 2018, Pages 864–870, https://doi.org/10.1093/aje/kwx299

[3] Kfir Y Levy. The power of normalization: Faster evasion of saddle points. arXiv preprint arXiv:1611.04831, 2016. https://doi.org/10.48550/arXiv.1611.04831

[4] Zhang, B., Shi, H., & Wang, H. (2023). Machine Learning and AI in Cancer Prognosis, Prediction, and Treatment Selection: A Critical Approach. Journal of multidisciplinary healthcare, 16, 1779–1791. https://doi.org/10.2147/JMDH.S410301

[5] Khalil, M.; Khalil, A.; Ngom, A. A Comprehensive Study of Vision Transformers in Image Classification Tasks. arXiv e-prints 2023, arXiv:2312.01232. https://doi.org/10.48550/arXiv.2312.01232.

[6] Kaddoura, S., Chandrasekaran, G., Elena Popescu, D., & Duraisamy, J. H. (2022). A systematic literature review on spam content detection and classification. PeerJ. Computer science, 8, e830. https://doi.org/10.7717/peerj-cs.830

# 6 Appendix

## 6.1 Additional figures

Figure 5 depicts a run of GD initialized further away at $a_0 = -25$, $b_0 = -1$. As can be seen the algorithm does not improve anymore after ~120 iterations once it arrives at “the plateau”. This region has compared to the rest of the landscape very shallow.
gradients. This is since in this region the parameters form a curve which classifies half of the data correctly and the other half wrong. The loss produced in this vast parameter region does not change significantly since the logistic function approaches 0 and 1 asymptotically.

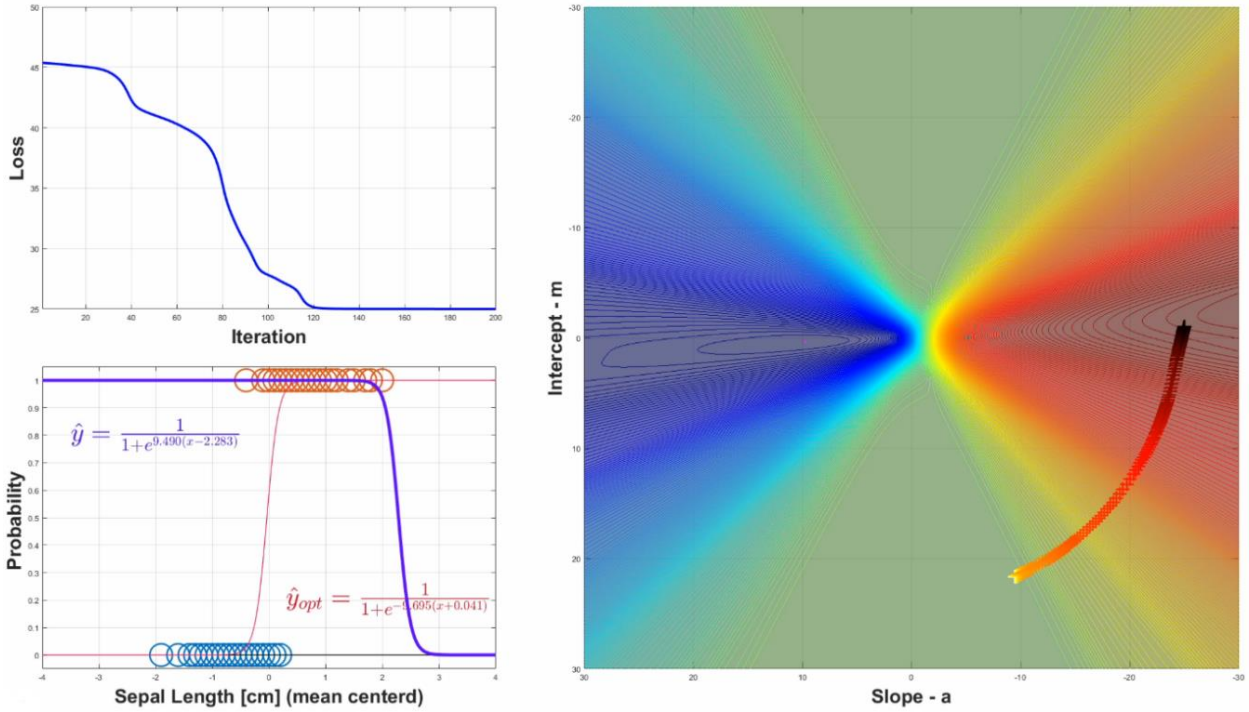



*Figure 5: Visualization of a GD run which gets stuck.*

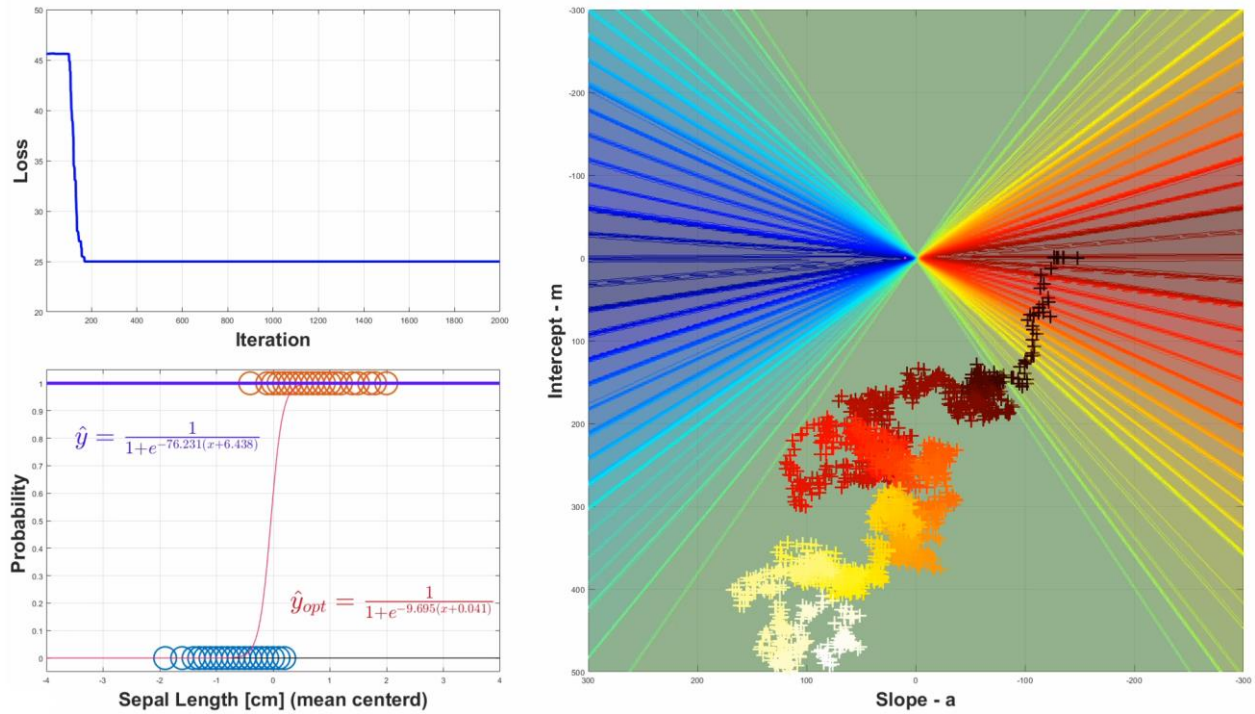



*Figure 6: Visualization of a Simualted Annealing run which did not converge. It takes sometimes very long to traverse the plateau.*

## 6.2 Introduction to classification with logistic regression

In machine learning, classification is the task of categorizing data into distinct classes based on specific features or attributes. This process enables algorithms to make informed decisions, predict outcomes, and automate tasks efficiently. It holds significant importance across a wide range of applications. For example, cancer diagnosis based on biomarkers[4], image classification and computer vision[5] or simply the classification of emails into spam vs. non-spam mails[6].

Logistic regression is such a classification algorithm. Despite its name, it is not only a regression model for predicting continuous outcomes, but it is also used for classification into discrete classes by thresholding the function. A core idea in classification is to convert a scalar prediction or logit into a probability for each class. The observation is then assigned the class with the highest probability.
To convert an n-dimensional observation, $\mathbf{x} = [x_1 \quad x_2 \quad \dots \quad x_n]$, into a probability for a class $i$, $P(y = i \mid \mathbf{x})$, the SoftMax function, also known as normalized exponential function, is used:

$$P(y = i \mid \mathbf{x}) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \qquad \text{Eq. 9}$$

This function maps any input to a value between 0 and 1 with the sum over all classes being 1 and where, $z_i = \beta_0^i + \beta_1^i x_1 + \beta_2^i x_2 + \cdots + \beta_n^i x_n = \boldsymbol{\beta}_\mathbf{i}^\mathrm{T}\mathbf{x}$, is the weighted sum of the n-dimensional input vector with the parameters that have to be estimated or "learned" based on labelled data. $K$ is the number of classes.
The case, where the outcome belongs to one of two classes, is called binary classification. Because the probabilities over all classes must sum up to 1, we obtain the probability of one class given the other.

$$P(y = 1 \mid X) = \frac{e^{z_1}}{e^{z_1}+e^{z_2}} \cdot \frac{e^{-z_1}}{e^{-z_1}} = \frac{1}{1+e^{z_2}e^{-z_1}} = \frac{1}{1+e^{\tilde{z}}} \qquad \text{Eq.10}$$

$$P(y = 0 \mid X) = \frac{e^{z_2}}{e^{z_1}+e^{z_2}} \cdot \frac{e^{-z_2}}{e^{-z_2}} = \frac{1}{e^{z_1}e^{-z_2}+1} = \frac{1}{1+e^{-\tilde{z}}} \qquad \text{Eq.11}$$

By setting a threshold, typically 0.5, logistic regression classifies observations or instances into one of the two classes based on whether the predicted probability exceeds the threshold.

The goal of the optimization is to find the parameters, also called weights, for each class such that the classification accuracy is as high as possible, meaning achieving as much correctly classified observations as possible. To solve this optimization problem, Maximum Likelihood Estimation has been established as the gold standard. Specifically, the cross-entropy loss function is used to quantify the performance of the model.

$$\mathcal{L}(\boldsymbol{\beta}) = -\sum_{i=n}^{N}\sum_{j=1}^{K} \delta_{\mathrm{j,y_i}} \log\big(P(y_i = j \mid \mathbf{x_i})\big) \qquad \text{Eq.12}$$

Where, $N$ is the number of observations, $K$ is the number of classes and $\delta_{\mathrm{j,y_i}}$ is the Kronecker delta which amounts to 1 if $\mathrm{y_i} = j$ and 0 otherwise. The Kronecker delta can be seen as a switch to activate the function only when the observed class matches the class to predict.

If there are only two classes to predict the cross-entropy loss can be written as:

$$\mathcal{L}_B(\boldsymbol{\beta}) = -\sum_{i=n}^{N} y_i \log\big(P(y_i = 1 \mid X_i)\big) + (1 - y_i) \log\big(P(y_i = 0 \mid X_i)\big) \qquad \text{Eq.13}$$

It has been proven that using the binary cross-entropy loss as objective function, it will result in a convex optimization problem no matter the number of input dimensions.[] This is the main reason why it is so useful to estimate model parameters.

Nevertheless, one can also use a least squares method. However, the resulting loss landscape will likely be non-convex, and it will be therefore much harder to find the optimal parameters for the model.

## 6.3 Code

The code to run the optimization algorithms described in this report can be found in the ZIP-File which contains this report. There are three Python files. A file which contains the necessary functions called “FunLibLogReg.py”. These functions are used by the other two files which produce results and graphics for the corresponding optimization algorithm. A MATLAB file is also included which was used to create the graphics for this report.